Spring Framework

Training Assignment

Document Code	**25e-BM/HR/HDCV/FSOFT**
Version	**1.1**
Effective Date	**20/11/2020**

Hanoi, /2024

RECORD OF CHANGES

No	Effective Date	Change Description	Reason	Reviewer	Approver
1	26/Sep/2024	Create a new assignment	Create new		

Contents

CODE	:	JSFW_S_A102
TYPE	:	SHORT
LOC	:	N/A
DURATION	:	120 minutes

Objectives:

After completing this assignment, trainees will:

1. Understand how to implement file upload and download functionality in a Spring Boot application.
2. Use REST APIs to upload and download files.
3. Create a file upload form using Thymeleaf and integrate it with the Spring Boot backend.
4. Handle file storage and serve files back as downloadable resources.

Problem Description:

We are continuing to build our new Learning Management System (LMS). This assignment focuses on the **assessment system**. A key feature for instructors is the ability to upload assessment materials (e.g., assignment briefs as PDFs, instruction files, or data files) for a course.

You will create a page where an instructor can fill in a "Title" and "Description" for an assessment and upload the corresponding file, all in one form. The page will also display a list of all previously uploaded materials, with links to download them.

Task 1: Create a File Upload REST API

Create a simple model class AssessmentMaterial.java in the model package. This class will just hold metadata about the uploaded file. It should contain:

- long id (You can use a simple counter or UUID to generate this)
- String title
- String description
- String fileName (This will store the name of the file on the server)

(Note: We will use an in-memory List<AssessmentMaterial> to store this data for now, rather than a full database.)

Task 2: File Service for Handling Storage Logic

Create a service class StorageService.java that will handle the logic of:

1. Storing an uploaded MultipartFile to a specific directory (e.g., ./uploads).
2. Loading a file as a Resource from that directory, given a filename.
3. (Optional) Initializing the storage directory on startup.

Task 3: Create a Controller for Handling Assessment Uploads

Create a controller class AssessmentController.java in the controller package with the following endpoints:

1. GET /assessments: This method should retrieve the list of all AssessmentMaterial objects (from your in-memory list) and display the main upload page.

2. POST /assessments/upload: This method must accept a MultipartFile *and* the title and description fields from the form. It will:
    - Call the StorageService to save the file.
    - Create a new AssessmentMaterial object with the title, description, and the file's name.
    - Save this new object to your in-memory list.
    - Redirect back to /assessments.
3. GET /assessments/download/{filename}: Serves the uploaded files for download by filename, using the StorageService.

Task 4: Create a Thymeleaf File Upload Form

Create a Thymeleaf template assessments.html in resources/templates/. This page must contain:

1. A form with enctype="multipart/form-data" and method="post" that posts to /assessments/upload.
2. An input field for title (<input type="text" name="title">).
3. A text area for description (<textarea name="description"></textarea>).
4. A file input (<input type="file" name="file">).
5. A submit button.

Task 5: Add File Download Links in Thymeleaf

In the *same* assessments.html template, *below* the upload form, add a section to display all uploaded materials.

- The controller's GET /assessments method must add the List<AssessmentMaterial> to the Model.
- In Thymeleaf, use a th:each loop to iterate over the list.
- For each item, display its title, description, and a download link (<a> tag) that points to /assessments/download/{filename} (e.g., th:href="@{/assessments/download/{fn}(fn=${material.fileName})}").

Task 6: Error Handling

Add error handling for cases such as file storage failures. If a file fails to upload, display an appropriate message to the user (e.g., using RedirectAttributes to add a message on redirect).

Technical Requirements:

- **JDK 1.8+**
- **Spring Boot 2+**
- **Maven 3+**
- **Thymeleaf**
- **Spring Boot Starter Web** for building REST APIs.
- **MultipartConfigElement** to handle multipart (file upload) requests.

-- THE END --